

Flashcards & Preguntas

Ingeniería de Software 1

Preparación para el Primer Parcial

Nota: la respuesta está debajo de cada pregunta (en verde). Tapala con una hoja o el dedo y probate a vos mismo.

Parte 1 — Flashcards rápidas (definiciones)

Preguntas cortas para repasar conceptos. Tapá la respuesta e intentá responder.

Entendimiento del problema / Discovery

Pregunta 1: ¿Cuál es la diferencia entre Output y Outcome?

✓**Respuesta:** **Output** es el entregable (lo construido, ej: un dashboard). **Outcome** es el efecto real: el cambio de comportamiento que se quería generar. El éxito se mide por el outcome, no por el output.

Pregunta 2: ¿Qué es el POV (Point of View) en Design Thinking?

✓**Respuesta:** Técnica para enmarcar el problema con el formato: *usuario → necesita → problema → porque (insight)*. Describe las necesidades del usuario y lo que piensa/hace ahora.

Pregunta 3: ¿Qué buscan los mapas de empatía?

✓**Respuesta:** Visualizar al usuario en 5+ áreas: **piensa, siente, dice, hace, escucha, dolores y necesidades**. Profundiza en pensamientos, emociones y comportamientos.

Pregunta 4: ¿Qué es un MVP y cuáles son sus 3 etapas de evolución?

✓**Respuesta:** **MVP** (Minimum Viable Product): versión más simple que valida la idea con el menor esfuerzo. Evolución: **ETP** (Testable, no funciona realmente) → **EUP** (Usable, lo mínimo funcional) → **ELP** (Lovable, que los usuarios podrían amar).

Pregunta 5: ¿Qué significa "Discovery como Dual Track"?

✓**Respuesta:** Que el Product Discovery NO se da solo al inicio, sino en paralelo al desarrollo. Se exploran nuevas ideas y se valida mientras se construye.

Clean Code

Pregunta 6: ¿Qué dice la regla del Boy Scout?

✓**Respuesta:** Dejar el código **más limpio** de lo que lo encontramos.

Pregunta 7: ¿Qué son los tests F.I.R.S.T?

✓**Respuesta:** **F**ast (rápidos), **I**ndependent (independientes entre sí), **R**epeatable (misma entrada → mismo resultado), **S**elf-validating (no requiere leer logs), **T**imely (oportunos, antes/durante la implementación, TDD).

Pregunta 8: ¿Por qué se recomienda reemplazar flags (booleans por parámetro) en las funciones?

✓**Respuesta:** Porque rompen el principio de única responsabilidad (SRP): la función hace cosas distintas según el flag. Se recomienda usar **polimorfismo o funciones separadas** en su lugar.

SOLID

Pregunta 9: ¿Qué principio viola un Stack que hereda de Vector?

✓**Respuesta:** **Liskov Substitution Principle (LSP)**. Si el Stack hereda de Vector, al tratar de indexar el stack se genera comportamiento indeseado. Un Stack no debería poder reemplazar a un Vector sin romper el código.

Pregunta 10: ¿En qué principio SOLID se basa decir "las clases dependen de abstracciones, no de concretas"?

✓**Respuesta:** **Dependency Inversion Principle (DIP)**. Si una clase depende de una interfaz, se puede cambiar fácilmente la clase concreta de fondo.

Pregunta 11: ¿Cuáles son las 4 características de un mal diseño?

✓**Respuesta:** **Rigidez** (cambio laborioso), **Fragilidad** (cambio rompe otras cosas), **Inmovilidad** (no se puede reusar), **Viscosidad** (hacks en vez de refactor).

Pregunta 12: ¿Cuál es la causa principal de un mal diseño?

✓**Respuesta:** Dependencias incorrectas entre módulos.

Agilidad / Scrum

Pregunta 13: ¿Qué son los 5 dominios del marco Cynefin?

✓**Respuesta:** **Claro** (causa-efecto obvia, mejores prácticas), **Complicado** (expertos, buenas prácticas), **Complejo** (causa-efecto en retrospectiva, prácticas emergentes), **Caótico** (acción inmediata, prácticas novedosas), **Desordenado** (no se sabe en qué dominio se está).

Pregunta 14: ¿Cuál es la diferencia entre Burndown Chart y Velocity Chart?

✓**Respuesta:** **Burndown:** mide si llegamos a tiempo dentro del sprint actual (story points restantes vs días). **Velocity:** mide la capacidad sostenida del equipo a lo largo de varios sprints (compromiso vs completado por sprint).

Pregunta 15: ¿Qué mide el Planning Poker?

✓**Respuesta:** Esfuerzo RELATIVO (basado en complejidad + tamaño), **NO tiempo ni complejidad pura**. Se mide en story points con consenso (cartas Fibonacci).

Pregunta 16: ¿Cuáles son las 3 preguntas del Daily Scrum?

✓**Respuesta:** 1) ¿Qué hice ayer que ayudó al objetivo del sprint? 2) ¿En qué trabajaré hoy para contribuir? 3) ¿Algo me bloquea a mí o a un compañero?

Pregunta 17: ¿Cuáles son las 4 salidas del Sprint Planning?

✓**Respuesta:** 1) Sprint Goal, 2) Sprint Backlog completo, 3) Plan inicial, 4) Noción de impedimentos.

Historias de usuario

Pregunta 18: ¿Qué significa INVEST?

✓**Respuesta:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Pregunta 19: ¿Qué son las 3 Cs de las historias de usuario?

✓**Respuesta:** **Card** (intención en tarjeta), **Conversation** (comunicación entre interesados), **Confirmation** (definición de criterios de aceptación).

Pregunta 20: ¿Cuál es el formato de una user story y el formato de su criterio de aceptación?

✓**Respuesta:** **User story:** "Como <rol>, quiero <funcionalidad>, para <beneficio>." **Criterio de aceptación:** "Dado que <contexto>, cuando <evento>, entonces <consecuencia>."

Pregunta 21: ¿Cuál es la diferencia clave entre User Story Mapping y Impact Mapping?

✓**Respuesta:** **User Story Mapping:** agrupa por funcionalidades (actividades principales, pasos, tareas). **Impact Mapping:** se orienta a los objetivos del negocio (Objetivo → Actores → Impactos → Entregables).

Modelo de dominio

Pregunta 22: ¿Cuáles son los 3 grupos de patrones de colaboración?

✓**Respuesta:** 1) **Genérico-específico** (actor-rol, item-item específico); 2) **Entero-parte** (gran lugar-lugar, ensamble-parte, contenedor-contenido, grupo-miembro); 3) **Específico-transacción** (transacciones simples y complejas, transacción cronológica).

Pregunta 23: ¿Cuáles son los 5 tipos de reglas de negocio?

✓**Respuesta:** **Tipo** (medicamento en container refrigerado), **Multiplicidad** (pallet hasta 10 cajas), **Propiedad** (tarjeta de crédito válida), **Estado** (orden cancelada no se entrega), **Conflicto** (alcohol + menor de edad).

Pregunta 24: ¿Qué aporta el Line Item en una transacción compuesta?

✓**Respuesta:** Agrega **multiplicidad**. Solo existe si se relaciona con al menos una transacción compuesta. Se asocia con un ítem específico.

UML

Pregunta 25: ¿Para qué sirven los estereotipos <<>> en UML?

✓**Respuesta:** Son un **mecanismo de extensión** para inyectar propiedades o especificar semánticas. Ej: <<component>>, <<interface>>, @RestController en Spring Boot.

Pregunta 26: ¿Cuál es la jerarquía en un diagrama de despliegue?

✓**Respuesta:** **Device** (hardware físico, ej PC) → **Execution Environment** (entorno lógico, ej Node.js, browser) → **Component** (el módulo que se ejecuta, ej web store).

Pregunta 27: ¿Qué diferencia hay entre agrupar paquetes por "capa técnica" y por "funcionalidad"?

✓**Respuesta:** **Por capa técnica** (controller/service/repository): *mala práctica*. Crea dependencias cruzadas y mala separación de responsabilidades. **Por funcionalidad** (loan, user, book): *buena práctica*. Cada módulo contiene lo que necesita y se interactúa por interfaces claras.

Atributos de calidad

Pregunta 28: ¿Por qué se rediseñan los sistemas?

✓**Respuesta:** No por definiciones funcionales, sino por **violar atributos de calidad**: mantenimiento, portabilidad, escalabilidad, performance, seguridad, interfaces obsoletas.

Pregunta 29: Nombrá 3 conflictos entre atributos de calidad

✓**Respuesta:** **Seguridad vs Usabilidad** (validaciones rigurosas dificultan acceso), **Portabilidad vs Performance** (optimizaciones específicas rompen portabilidad), **Seguridad vs Confiabilidad** (ante ataque detener sistema reduce disponibilidad).

Pregunta 30: ¿Cuál es la diferencia entre escalabilidad vertical y horizontal?

✓**Respuesta:** **Vertical:** agregar recursos físicos (memoria, CPU, disco) a la infraestructura existente.
Horizontal: incrementar el número de computadoras para dividir la carga.

Arquitectura y patrones

Pregunta 31: ¿Cuáles son las 4+1 vistas de Kruchten y qué representa cada una?

✓**Respuesta:** **Lógica** (requisitos funcionales, abstracción, para desarrolladores), **Desarrollo/Componentes** (organización de módulos), **Procesos** (requisitos no funcionales: rendimiento, concurrencia), **Física/Despliegue** (distribución en hardware), **Escenarios** (casos de uso que unifican las otras 4).

Pregunta 32: En Clean Architecture, ¿cuál es la regla de dependencias?

✓**Respuesta:** Las dependencias apuntan **hacia el centro**. Entities NO dependen de nada. Use Cases solo dependen de Entities. Los Adapters dependen de Use Cases. Los Frameworks/Drivers son la capa más externa.

Pregunta 33: ¿Qué patrón de arquitectura tiene un "núcleo" de reglas de negocio aislado por puertos y adaptadores?

✓**Respuesta:** **Arquitectura Hexagonal** (también llamada Puertos y Adaptadores).

REST

Pregunta 34: ¿Por qué REST es stateless y qué ventajas tiene?

✓**Respuesta:** Cada request es independiente y contiene TODA la info necesaria; no se guarda contexto entre peticiones. **Ventajas:** mayor escalabilidad, más fácil de entender, permite caches eficientes.

Pregunta 35: ¿Cuál es la diferencia entre PUT y PATCH?

✓**Respuesta:** **PUT** reemplaza el recurso completo. **PATCH** modifica parcialmente (no pisa, actualiza solo los campos enviados).

Pregunta 36: ¿Qué significa HATEOAS?

✓**Respuesta:** **Hypermedia As The Engine Of Application State**. Las respuestas contienen hipervínculos a recursos relacionados, y el cliente descubre recursos dinámicamente sin conocer URLs de antemano.

Pregunta 37: ¿Cuál es la estructura de un JWT?

✓**Respuesta:** `Header.payload.signature`. **Header** (algoritmo y tipo, encodeado base64), **Payload** (metadata del user, encodeado base64), **Signature** (hash de header+payload+clave privada del server, CIFRADO).

Pregunta 38: ¿Cuál es la diferencia entre JWT y Opaque Token?

✓**Respuesta:** **JWT:** codifica info (uid, role, expiración). Se valida SIN consultar la base de datos. Mantiene stateless. **Opaque Token:** string aleatoria sin info legible. Requiere consultar la DB para validarlo.

Pregunta 39: ¿Para qué sirve un Refresh Token?

✓**Respuesta:** Permite obtener un nuevo access token sin que el usuario tenga que volver a ingresar sus credenciales. El access token tiene tiempo de vida corto; el refresh token permite renovarlo.

Pregunta 40: ¿Por qué NUNCA se debe pasar un API Key en la URL?

✓**Respuesta:** Porque los navegadores guardan las URLs en el historial, se registran en los logs del servidor, y pueden quedar expuestas en headers Referer. Se pasa por **header** (X-API-Key) o en el body, nunca en la URL.

Parte 2 — Preguntas tipo parcial (desarrollo)

Preguntas de desarrollo al estilo del parcial real. Tratá de resolverlas antes de leer la respuesta modelo.

Ejercicio 1 — Estimaciones en Planning Poker

Enunciado: Un equipo de desarrollo de tres integrantes se encuentra trabajando en un proyecto ágil, utilizando la técnica de poker planning y utilizando la secuencia de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987) para asignar peso a las tareas. Al comenzar la estimación de tareas se encuentran con el siguiente conflicto: Uno vota que una tarea debe recibir el peso de 144, sin embargo, los otros dos uno vota que debe ser de 233 y el otro 89. En la siguiente les pasa lo mismo, pero con los valores 3, 5 y 8. Eso les pasa en casi todas las tareas, y les cuesta llegar a un consenso. Pasan mucho tiempo analizando y discutiendo las estimaciones. Más a o menos están alineados con la dificultad, pero no suelen asignar todos el mismo peso. ¿Qué les recomendaría?

Respuesta modelo

Varias recomendaciones combinadas:

- **Reforzar la conversación ANTES de estimar:** en el sprint planning, si hay dudas sin respuestas inmediatas, el equipo debe definir suposiciones explícitas ("asumamos que no se pide CAPTCHA en esta versión"). Esto reduce el rango de interpretación y las diferencias grandes se achican.
- **Entender que la estimación es RELATIVA:** el Planning Poker NO busca precisión absoluta, busca esfuerzo relativo (complejidad + tamaño). Que uno vote 144 y otro 233 en tareas grandes es esperable: ambos están viendo que es "muy grande", lo cual ya es información útil. No hay que pasar mucho tiempo discutiendo pesos grandes.
- **Recortar la escala usada:** para tareas del orden de cientos de story points, la historia debería DIVIDIRSE en tareas más pequeñas (por ej. ≤ 13 o ≤ 21 puntos). Una tarea de 144+ es casi siempre una épica disfrazada de historia, y allí los votos se dispersan naturalmente.
- **Usar T-shirt sizes para primera pasada:** si el equipo se traba mucho, hacer una estimación inicial rápida con tallas (S, M, L, XL) y recién después afinar con Fibonacci. Así se evita perder tiempo en números altos.
- **Debatir solo las diferencias GRANDES:** cuando dos votan 3 y 5, y uno vota 8, puede tomarse el consenso sin re-votar (son adyacentes en Fibonacci). Re-votar tiene sentido cuando las diferencias son mayores (ej. 3 vs 13).
- **Considerar que la estimación es para compromiso REALISTA, no para exactitud:** el objetivo es que el equipo pueda comprometerse a un volumen de trabajo alcanzable en el sprint, no acertar un número exacto.

Ejercicio 2 — Diseño de API REST para app deportiva

Enunciado: Imaginen el contexto de una app mobile, donde los usuarios se pueden bajar una aplicación móvil Android o iOS, y también pueden acceder vía web a una aplicación online.

La app permite buscar eventos de tipo deportivo relacionados a running y ciclismo. Se pueden buscar y filtrar los eventos por nombre, tipo, fecha, lugar (paginado). También permite crear una cuenta de usuario y loguearse a la app con sus credenciales. Un usuario debe tener por lo menos nombre, email, apellido, y opcionales foto avatar, fecha de nacimiento y género. Estando logueado puede marcar y desmarcar como favorito los eventos que le interesan.

También existe un rol administrador, que desde la web, puede dar de alta nuevos eventos, indicando nombre, tipo, fecha, y lugar así como un detalle y una foto para el mismo. También puede editarlos.

a) Diseño de la interfaz de una API REST

Con JWT como bearer token y refresh token simple:

Autenticación

POST /api/v1/auth/token

Headers: Authorization: Basic <base64(email:password)>

Response 200: { "access_token": "<JWT>", "refresh_token": "<opaque>" }

Response 401 si credenciales incorrectas

POST /api/v1/auth/refresh

Headers: Authorization: Bearer <refresh_token>

Response 200: nuevo access_token

Usuarios

POST /api/v1/users — crear cuenta (registro)

Body: { "name", "email", "lastName", "avatar"?, "birthDate"?, "gender"? }

GET /api/v1/users/me — perfil del usuario logueado

PATCH /api/v1/users/me — modificar perfil (parcial)

Eventos (lectura pública)

GET /api/v1/events?type=running&dateFrom=2025-07-01&location=CABA&page=1&limit=20

Response incluye: { "data": [...], "meta": {page, per_page, page_count, total_count}, "links": {...} }

GET /api/v1/events/{eventId} — detalle

Eventos (admin)

POST /api/v1/events — requiere JWT con rol admin

PATCH /api/v1/events/{eventId} — modificar

DELETE /api/v1/events/{eventId}

Subida de foto: POST /api/v1/events/{eventId}/photo con Content-Type multipart/form-data.

Favoritos

GET /api/v1/users/me/favorites

PUT /api/v1/users/me/favorites/{eventId} — marcar favorito (idempotente, por eso PUT)

DELETE /api/v1/users/me/favorites/{eventId} — desmarcar

Headers típicos en todas las requests autenticadas:

Authorization: Bearer <JWT>

Accept: application/json

Accept-Encoding: gzip

b) Despliegue de API y app store — ¿Qué problemas pueden surgir?

Problema principal: NO podemos garantizar que los usuarios se instalen la última versión de la app. Al publicar en App Store / Play Store, los usuarios actualizan en momentos distintos (y algunos nunca actualizan).

Problemas concretos:

- Versiones viejas de la app consumiendo la API nueva y rompiéndose (si hay cambios incompatibles).
- Usuarios sin conexión por tiempo largo no ven features nuevas.
- Dispositivos viejos que no soportan la última versión del SO y se quedan atrás.

Soluciones:

- **Versionado de la API:** /api/v1/, /api/v2/. Cuando hay cambio incompatible, creamos una nueva versión y mantenemos la vieja hasta que la mayoría de los usuarios migre.
- **Cambios compatibles hacia atrás siempre que se pueda:** agregar campos opcionales en lugar de modificar existentes. Agregar nuevos endpoints en vez de cambiar contratos.
- **Forzar actualización para cambios críticos:** el cliente manda su versión en un header custom (ej. X-App-Version), y si es demasiado vieja la API responde con un código que haga que la app muestre "actualizá para continuar".
- **Deprecación gradual:** marcar endpoints como deprecated con header Deprecation y Sunset, y comunicar plazos.
- **Feature flags:** activar features desde el backend sin requerir nueva versión de la app.

c) Cambio compatible vs no compatible — ejemplos

Cambio COMPATIBLE hacia atrás (ejemplo):

Agregar el campo opcional "distancia_km" a la respuesta de GET /events/{id}. Las apps viejas simplemente ignoran el campo, las nuevas lo muestran.

Cambio NO compatible (ejemplo):

Cambiar el formato del campo "date" de string ISO a número de timestamp Unix, o renombrar "type" a "sport_type".

¿Por qué NO es compatible?

Porque una app vieja espera un string y recibiría un número (o viceversa), causando un crash o mal parseo. Renombrar un campo significa que la app vieja buscaría una propiedad inexistente.

Cómo solucionarlo si llegan requests viejas:

- Crear un nuevo endpoint o versión (/api/v2/events/{id}) con el formato nuevo.
- Mantener el endpoint viejo (/api/v1/events/{id}) devolviendo el formato viejo.
- Si la app vieja pega a /api/v1/, le respondo en formato viejo. Si pega a /api/v2/, en formato nuevo.
- Eventualmente deprecatear v1 y forzar a los usuarios a actualizar.

Ejercicio 3 — Análisis de code smell y extensión

Enunciado:

Se presentan dos clases: JSONDataProcessor y XMLDataProcessor. Cada una implementa un método process(fileName) que: muestra un println, lee el archivo, parsea (JSON o XML), valida, si es válido extrae contenido y lo procesa, si no lanza excepción, lo guarda, guarda metadata, y termina con otro println.

a) Code smells detectados

- Violación DRY: las dos clases tienen EXACTAMENTE el mismo flujo (solo cambian read/parse/validate/extract/save).
- Violación OCP: si mañana quieren agregar CSV, hay que crear otra clase que repita todo el flujo.
- Violación SRP parcial: el método process() mezcla varias responsabilidades (leer, parsear, validar, extraer, guardar, loggear).
- Manejo de errores con IllegalArgumentException genérico: debería ser una excepción específica del dominio.
- println directo en la lógica de negocio: rompe la separación con la capa de presentación/logging.

b) Cómo agregar CSV respetando SOLID

Aplicar el patrón Template Method + Strategy, invirtiendo la dependencia hacia una interfaz:

```
interface DataFormatHandler {
    Object parse(String content);
    boolean validate(Object parsed);
    Object extractContent(Object parsed);
    void saveProcessed(Object content, String fileName);
    void saveMetadata(Object content);
}

class DataProcessor {
    private final DataFormatHandler handler;
    private final FileReader reader;
    private final Logger logger;
    public DataProcessor(DataFormatHandler handler, FileReader reader, Logger logger) {
        this.handler = handler;
        this.reader = reader;
        this.logger = logger;
    }
    public void process(String fileName) {
        logger.info("Loading data...");
        String content = reader.read(fileName);
        Object parsed = handler.parse(content);
        if (!handler.validate(parsed))
            throw new InvalidContentException("Contenido inválido");
        Object extracted = handler.extractContent(parsed);
        handler.saveProcessed(extracted, fileName);
        handler.saveMetadata(extracted);
        logger.info("Processing complete.");
    }
}

class JsonHandler implements DataFormatHandler { ... }
```

```
class XmlHandler implements DataFormatHandler { ... }  
class CsvHandler implements DataFormatHandler { ... } // NUEVO, sin tocar DataProcessor
```

Ventajas de este diseño:

- **DRY:** la lógica de process() está una sola vez.
- **OCP:** agregar CSV = crear una nueva clase, no modificar nada existente.
- **SRP:** DataProcessor solo orquesta; cada handler solo conoce su formato.
- **DIP:** DataProcessor depende de la abstracción DataFormatHandler, no de JSON/XML concretos.
- **LSP:** cualquier handler (CSV, YAML, Parquet) puede reemplazar a otro sin romper DataProcessor.
- **ISP:** la interfaz tiene solo lo que los handlers realmente usan.
- **Testeable:** se puede mockear el FileReader, el Logger y el Handler para testear DataProcessor aislado.

Estrategia para el día del parcial

Antes del parcial

- Repasar flashcards 1-2 días antes, no la noche anterior.
- Hacer un ejercicio de diseño de API completo (como el ej. 2).
- Redibujar los 3 grupos de patrones de colaboración en una hoja en blanco.
- Practicar explicar cada SOLID con un ejemplo propio de 2 líneas.

Durante el parcial

- Leer TODO el parcial antes de empezar (5 minutos). Así te formás una idea de la dificultad relativa.
- Empezar por lo que mejor sabés para construir confianza.
- En preguntas de diseño: primero esbozá a lápiz, después prolijamente.
- En código con code smells: identificá violaciones de SOLID, nombrá el patrón aplicable (Strategy, Template Method, etc.) y mostrá el código mejorado.
- En diseño de API: no olvides pagination, auth (Bearer), versionado en URL, status codes correctos.

Errores comunes a evitar

- Confundir estimación con predicción de tiempo (es esfuerzo relativo).
- Usar GET para enviar credenciales (siempre POST para login).
- Poner API keys en la URL.
- Olvidar que HATEOAS implica links dinámicos en las respuestas.
- Decir "Burndown mide la velocidad del equipo" (eso es Velocity; Burndown mide si llegamos al sprint).
- En Clean Architecture: invertir la dirección de dependencias (deben apuntar al centro, no al revés).

¡Vas a romperla, Tomas! 